

Urban Service Resolution Intelligence Platform

A Machine Learning–Driven Decision-Support System for Municipal Service Request Prioritization

Project Overview

Domain	Civic Technology / Urban Analytics / Applied Machine Learning
Dataset	NYC 311 Service Requests (official open dataset; ~1 year, ~300K–1.5M records used)
Core ML Task	Regression — prediction of service request resolution time (hours)
Evaluation Metric	Mean Absolute Error (MAE), benchmarked against category-median baselines
Delivery Stack	PostgreSQL, REST API, Streamlit, Power BI / Tableau, HTML/CSS/JS front end

1. Introduction

Municipal governments receive an enormous volume of citizen service requests every day — covering issues such as garbage collection, street conditions, noise complaints, water leaks, and sanitation. Operations staff cannot manually inspect or track every request, which means resources are often allocated reactively rather than strategically. The Urban Service Resolution Intelligence Platform is a full-stack, machine-learning-powered decision-support system designed to answer a more actionable question than “can we predict something”: it identifies which requests are likely to remain unresolved for a long time, where the city’s operational bottlenecks are concentrated, and where management attention should be directed first.

2. Problem Statement

Using New York City’s official 311 Service Request dataset (approximately 22 million records with 44 attributes, including creation date, closure date, agency, problem type, and geographic location), the project develops a system that predicts the expected resolution time for a newly logged service request and flags high-delay-risk cases. To keep the project within a realistic academic scope while remaining representative, a curated subset of roughly 300,000 to 1.5 million records — spanning one year and a set of major service categories — will be used. A critical design constraint is the avoidance of data leakage: the model is trained only on information available at the time a request is created, and never on closure dates or other fields derived from the future.

3. Objectives

- Predict service-request resolution time using engineered features such as problem category, agency, borough, day/hour of creation, and historical category/agency workload.

- Benchmark the ML model against two non-ML baselines (overall median and category-wise median) and report percentage improvement, including cases where ML does not outperform the baseline.
- Provide model explainability (e.g., SHAP) so operations managers understand which factors drive a given delay-risk prediction.
- Apply descriptive statistics, correlation analysis, and hypothesis testing (t-test, chi-square) to validate operational findings such as category-level or workload-driven delays.
- Deliver an interactive, role-based web application — Operations Manager, Analyst, and Field/Service Staff — supported by Streamlit and Power BI/Tableau dashboards.

4. Proposed System Overview

The platform is organized around three user roles. The Operations Manager monitors overall service performance, identifies slow categories and locations, and prioritizes incoming requests through a command-center dashboard. The Analyst explores historical patterns, compares agencies and categories, and investigates root causes of delay using statistical tooling. Field/Service Staff view assigned and prioritized requests, inspect location and request details, and update resolution status. The architecture follows a layered design: a JavaScript-based front end communicates through a REST API with a PostgreSQL database and the trained ML model, feeding an analytics layer that powers both the Streamlit operational dashboard and Power BI/Tableau analyst reports.

5. Methodology

- Data acquisition and cleaning of the NYC 311 dataset, restricted to a defined time window and selected major categories.
- Exploratory data analysis: distribution of resolution times (mean, median, mode, quartiles, standard deviation), correlation between workload and resolution time, and category/agency comparisons.
- Feature engineering from categorical, temporal, and historical-workload attributes, with strict exclusion of any post-creation (leakage) fields.
- Model development progressing from linear/regularized regression to tree-based ensembles (Decision Tree, Random Forest, Gradient Boosting), selected and justified in the accompanying SRS.
- Model evaluation using MAE against Level 0 (overall median) and Level 1 (category median) baselines, with SHAP-based explanation of feature importance.
- Integration into a full-stack web application with role-based dashboards and a geographic (map-based) view of request density and delay.

6. Expected Outcomes

The completed system will allow a manager to move from a high-level city-wide dashboard, into a geographic drill-down by borough and category, into a per-request delay-risk prediction with an explanation of contributing factors, and finally into a prioritized action queue ranking the most at-risk open requests. The anticipated business conclusion is that operational delay is not driven uniformly by overall request volume, but by a concentrated subset of categories and locations — an insight that supports targeted, rather than blanket, resource allocation.

7. Tools and Technologies

Python (pandas, scikit-learn, SHAP) for data processing and modeling; PostgreSQL for storage; a REST API layer connecting the database and model to the front end; Streamlit for the interactive operational application; Power BI or Tableau for analyst-facing reporting; and HTML/CSS/JavaScript for the web front end.

8. Significance

This project demonstrates an end-to-end applied data science and software engineering workflow — from raw open-government data to a deployed, explainable, decision-support product — combining regression modeling, statistical inference, geospatial visualization, and full-stack development within a single, resume-worthy civic-technology platform.

Name :- Priyanshu Kumar

University Roll_No :- 22515500074